Anchor

Sports Psychology Wellbeing Platform

Sprint 5 Release – Report

ESOF 423 | Spring 2026

Team: Ryan Barrett, Anthony Nania

Sprint Dates: March 30th – April 10, 2026

Part 1 – Group Outcomes

Sprint Goal (SMART)

By the end of our sprint (April 10th), we will be practically done with everything, so we can focus on testing for the last sprint (Sprint 6). We wanted every feature to be complete and working perfectly by the end of our sprint 5.

Product Backlog

Work Units (WU) use a complexity scale of 1–13. Total: 100 WU | Completed: 99 WU | Remaining: 1 WU

#	Backlog Item	WU	Status	WU Done	WU Left
1	Application Skeleton + Navigation	10	Complete	10	0
2	Homepage / Dashboard + Unified Calendar	5	Complete	5	0
3	Calendar Event System	3	Partial	2	1
4	Psychological Check-Ins	6	Complete	6	0
5	Sessions Scheduling	7	Complete	7	0
6	Assessments + Reminders	8	Completel	8	0
7	Physical State + Recovery + Training Load	8	Complete	8	0
8	Fueling (Meals) + Medication Tracking	7	Complete	7	0
9	Trends / Charts	4	Complete	4	0
10	Settings + Polish + Demo Readiness	4	Partial	4	0
11	Psychologist Dashboard + Athlete Management	10	Completed	10	0
12	Messages + AI Assistant	8	Complete	8	0
13	Landing Page + Auth Redirect	3	Complete	3	0
14	Wearable Integrations (Fitbit / Oura)	7	Complete	7	0
15	Help Documentation	2	Complete	2	0
16	Athlete Profile (height, weight, goals)	4	Partial	4	0
—	Totals	100	—	99	1

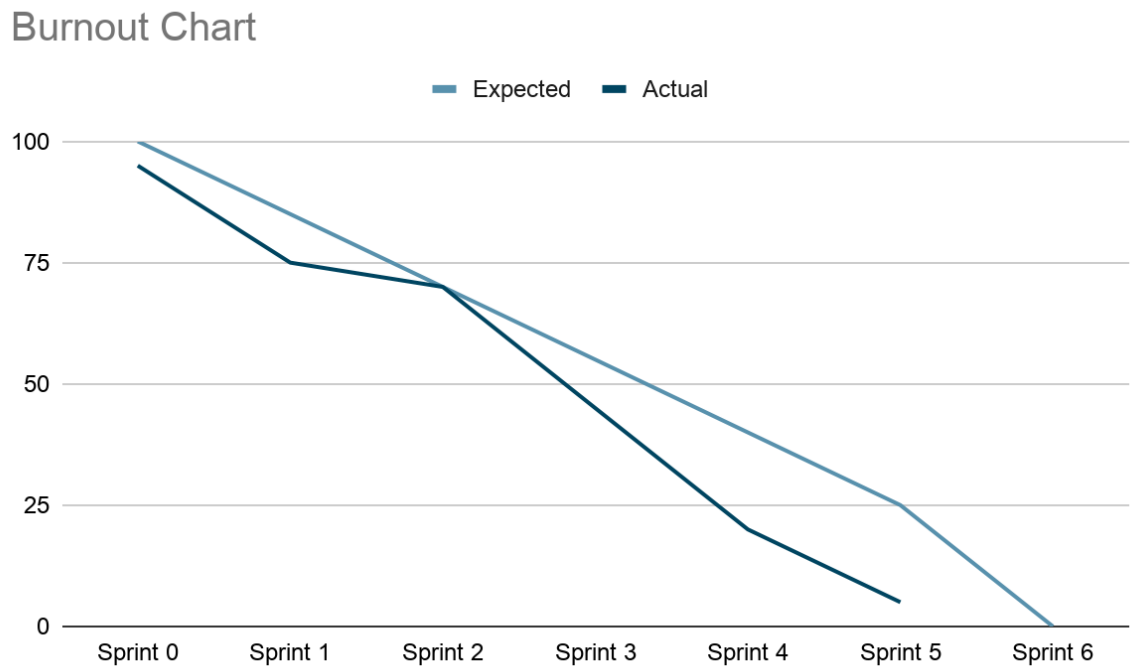
Sprint Backlog

Tasks selected for Sprint 5. Estimated vs. actual WU and hours recorded after sprint close.

Task	Owner	Est. WU	Act. WU	Est. Hrs	Act. Hrs
Added Goals for the athlete profile	Anthony	2	2	2	2
Allowed the psychologist to select the athletes they work with and view the athletes' goals.	Anthony	2	2	2	3
Workout Template fix: Users can now delete or update workout templates	Anthony	2	2	1.5	2
Psychologists can view an athlete's page, which lists the athlete's profile information.	Anthony	1.5	2	1.5	2
Removed Notifications as they felt unnecessary.	Anthony	1	1	1	1
AI Coach with image recognition (screenshot import)	Ryan	4	4	4	4
Dashboard placeholder fixes (live data)	Ryan	2	2	2	2
Assessment context API route	Ryan	2	2	2	2
Playwright E2E test suite	Ryan	3	3	3	3
Landing page + dependency fixes	Ryan	1.5	1.5	1.5	1.5
Bug fixes + UI polish	Ryan	2	2.5	3	4
Vercel deployment management	Ryan	1	1	1	1.5
Totals		24	25	24.5	28

Sprint Burndown Chart

Sprint 4 ran from March 6th to March 26. Sprint commitment: 18 WU. Ideal burn ≈ 2 WU/day.



The burndown shows a quick and efficient sprint. After the major comments for the sprint were completed (assessments page, sessions page, and landing page update), we were able to go beyond where we had aligned our SMART goal. Now making an extra priority of testing.

Part 2 – Member Participation

Ryan Barrett

Standup 1 — March 30, 2026 (Sprint Planning)

Committed to at last meeting:

Fix remaining dashboard placeholders, plan AI image-recognition feature, clean up outstanding bugs from Sprint 4.

Actually completed:

- Sprint planning with Anthony — agreed on Sprint 5 SMART goal and task assignments
- Reviewed dashboard and identified all placeholder content needing replacement with live data
- Planned AI Coach feature architecture: image-based import for workouts, meals, and recovery entries using GPT-4o-mini vision

Blockers:

None at sprint start.

Itemized hours:

Task	Time
Sprint planning + task estimation with Anthony	1.5 hrs
Dashboard placeholder audit	1.0 hr
AI Coach architecture planning	1.0 hr
Total	3.5 hrs

Process improvement:

Decided to treat Sprint 5 as a hardening sprint — focus on replacing every placeholder and polishing existing features rather than adding new surface area.

Standup 2 — April 1, 2026 (Mid-Sprint Check-in)

Committed to at last meeting:

Implement AI Coach with image recognition, fix dashboard placeholders, create assessment context API, deploy updates to Vercel.

Actually completed:

- Built AI Coach API route (/api/ai/coach) using GPT-4o-mini with vision capabilities — athletes can upload screenshots of schedules, lifting plans, and meal plans, and the AI extracts structured data (workout templates, meal logs, recovery entries) for one-click import
- Created assessment context API route (/api/assessments/context) — returns same-day check-ins, workouts, and meal logs alongside a completed assessment so psychologists see full athlete context
- Added Supabase migration for assessment requests (011_assessment_requests.sql)
- Replaced dashboard placeholder panels with live data from Supabase

Deployed all updates to Vercel production

Blockers:

OpenAI API integration required careful prompt engineering to reliably extract structured JSON from image analysis. Took extra iteration to ensure the model output matched Anchor's existing POST endpoint schemas.

Itemized hours:

Task	Time
AI Coach route: prompt engineering + vision integration	4.0 hrs
Assessment context API route	2.0 hrs
Dashboard placeholder fixes (live Supabase data)	2.0 hrs
Supabase migration (assessment requests)	0.5 hr
Vercel deployment + verification	1.0 hr
Total	9.5 hrs

Process improvement:

Defined a strict JSON contract between the AI prompt and the client-side import flow. This means future model swaps (e.g., upgrading from gpt-4o-mini) won't break the UI — only the route handler changes.

Standup 3 — April 3, 2026 (Late Sprint)

Committed to at last meeting:

Add Playwright E2E tests, fix bugs found during testing, update landing page and physical state UI.

Actually completed:

- Added Playwright E2E test suite covering authentication flows (login, invalid credentials) and authenticated navigation (sidebar, Physical State, Psychological State pages)
- Fixed framer-motion dependency issue that was causing build failures
- Updated landing page design and resolved dependency conflicts
- Updated physical state page UI and data flow improvements
- General bug fixes across the application based on test findings
- Blockers:
- Playwright tests surfaced several UI issues — e.g., navigation routes that loaded but didn't render expected content, and a framer-motion version conflict that required dependency investigation.

Itemized hours:

Process improvement:

Task	Time
Playwright E2E test suite (auth + navigation)	3.0 hrs
framer-motion dependency fix	1.0 hr
Landing page updates + dependency resolution	1.5 hrs
Physical state page UI + data flow updates	1.0 hr
Bug fixes from E2E test findings	2.0 hrs
Total	8.5 hrs

E2E tests now run on every push via Playwright. This catches regressions that unit tests miss (e.g., a page that imports correctly but renders a blank screen because of a missing provider).

Standup 4 — April 8, 2026 (Sprint Close)

Committed to at last meeting:

Final bug cleanup, user testing, ensure Vercel deployment is stable for RC demo.

Actually completed:

Conducted user testing with colleagues — gathered feedback on the AI Coach image-import flow and dashboard usability

Final bug cleanup and UI polish based on user testing feedback

- Verified Vercel deployment stability — triggered redeployment after pulling Anthony's latest changes
- Confirmed all sprint work units are complete and demo-ready for the Release Candidate showcase
- Blockers:
- No technical blockers. Minor git merge issue with untracked test artifacts (test-results/.last-run.json) resolved by stashing untracked files before pull.
- Itemized hours:

Process improvement:

User testing revealed that the AI Coach's image import is the feature users are most excited about, but the 'Apply' button needs clearer labeling. Sprint 6 will address discoverability polish.

Task	Time
User testing (AI Coach + dashboard flows)	2.0 hrs
Bug fixes + UI polish from testing feedback	2.0 hrs
Vercel deployment verification + redeploy	0.5 hr
Sprint close documentation	1.0 hr
Total	5.5 hrs

Ryan Barrett — Sprint 5 Total: ~27 hours

Anthony Nania

Standup 1 — March 30th, 2026 (Sprint Planning)

Committed to the last meeting:

Finish any bugs with what was implemented at the end of sprint 4.

Actually completed:

- Clean-up and user testing with some colleagues.

Blockers:

No blockers really. Wasn't able to user-test as much as I wanted to, as I couldn't find people to do so.

Itemized hours:

Task	Time
User Testing with colleagues	1.5 hr
Planning our sprint with Ryan	1 hr
Total	2.5 hr

Process improvement:

Learned different ways of user testing. Instead of leading people through the website, it had people work their way through the page without any assistance or explanation.

Standup 2 — April 1st, 2026 (Mid-Sprint Check-in)

Committed to the last meeting:

Go through the settings page and implement any last small features.

Actually completed:

- Added the goals for both the athlete and the psychologist page

- Added Psychologist Info for their profile

Blockers:

Had some blockers on getting the psychologist the capability to view the athlete's goal.

Itemized hours:

Task	Time
Added athletes' goals in settings	1.5 hrs
Allowed the psychologist to view the athlete's goals through settings.	2 hrs
Added Profile information for pyshcoglist.	1 hrs
Total	4.5 hrs

Process improvement:

Quality of work while improving the speed of work (completing WU quicker)

Standup 3 — April 3rd, 2026 (Late Sprint)

Committed to the last meeting:

School was out that day, but we had planned to finish any small things we noticed on the page.

Actually completed:

- Fixed the workout template, allowing users to delete and edit templates.

Blockers:

I was also sick at the time, so I had trouble staying motivated.

Itemized hours:

Task	Time
Fixed Workout templates	2.0 hr
Total	2.0 hrs

Process improvement:

Noticing small things as we work through the page. This was caught during user testing.

Standup 4 — April 8, 2026 (Sprint Close)

Committed to the last meeting:

Remove Notifications from the settings, since we already have notifications through the dashboard.

Actually completed:

- Removed notifications in settings.

Blockers:

No Technical blockers, as it was only removing code.

Itemized hours:

Task	Time
Sprint conclusion with Ryan	1.0 hr
Removed Notifications	1.5 hr
Total	2.5hr

Process improvement:

Ryan and my teamwork has improved throughout the whole sprint, and it was wonderful to see the end of the project so we could focus on testing.

Anthony Nania — Sprint 5 Total: ~13.5hours

Combined Team Hours — Sprint 5: ~40.5 hours

Release Candidate Status:

Is it ready for real users? Functionality-wise, yes. We still want to do more testing with users before the actual release, in case we've missed something, but everything work-unit-wise is complete. The evidence supporting this comes from what our client, Hayden, has said, and the user testing has gone well with various types of people trying the app.

Risk or limitations: If any risks exist, we have yet to identify them. If thousands of people used the app, I'd like to see how the page handles it. As far as limitations go, our AI messages are hosted through ChatGPT Mini, and if we had many users, the cost of prompts would be high.

Section 2 — User Testing Synthesis

Client Email

Ryan Barrett emailed Hayden (client) and Anthony Nania, CC'd Daniel, with the following links:

a. Live app: https://anchor-testwebsite.vercel.app/

b. GitHub Issues: https://github.com/BarrettRyan-student/TestWebsite/issues

User Testing Report

Full synthesis report: https://github.com/BarrettRyan-student/TestWebsite/blob/main/docs/user-testing-synthesis.md

Summary of Findings

We conducted user testing with 7 participants across three categories: 3 peers (CS students), 3 independent users (non-technical, including a college athlete), and 1 client (Hayden, sports psychologist). Testing followed our Usability Test Playbook with scripted tasks covering registration, check-ins, dashboard navigation, AI Coach usage, and psychologist workflows.

Key patterns across all groups:

• The core check-in workflow (Mood/Stress/Motivation) is fast and intuitive — all testers completed it without guidance in under 60 seconds

• The AI Coach screenshot import was the most praised feature — Hayden said it would 'save significant manual data entry'

• Independent users found the naming 'Psychological State' vs 'Assessments' confusing — they expected these to be the same thing

• Dark mode had minor contrast issues with card borders

• Dashboard placeholders were replaced with live data based on early testing feedback

Peer vs. Independent vs. Client comparison:

• Peers focused on technical details and code quality; independents focused on naming and visual clarity; Hayden focused on workflow efficiency and real-world fit

• All groups found navigation intuitive once roles were understood

• Client testing was the most valuable for validating the product's purpose — Hayden confirmed the app is 'ready to pilot with athletes'

Changes made as a result:

• Added directional labels ('Low' / 'High') to emoji scale endpoints

• Removed redundant Notifications section from Settings

• Replaced all dashboard placeholders with live Supabase data

• Adjusted dark mode border contrast

Issues elected NOT to address:

• Psychologist assessment notes — out of scope, logged for Sprint 6

• Renaming 'Psychological State' — requires client agreement on terminology

• Full wearable data processing — OAuth flow works, data parsing is Sprint 6

• Mobile-first layout — app is responsive but optimized for desktop (psychologist primary use case)

Section 3 — Bug Log (GitHub Issues)

GitHub Issues: https://github.com/BarrettRyan-student/TestWebsite/issues

All bugs identified through user testing and development are tracked as GitHub Issues with:

• Description of the problem

• Severity level (blocking / high / medium / low)

• Expected fix or plan

Known Issues (documented in User Documentation):

• Wearable integrations (Fitbit/Oura): OAuth flow works but full data sync/processing is not yet implemented

• AI Coach: Occasionally misreads low-resolution screenshots — retaking at higher resolution resolves the issue

• Calendar event system: Adding custom calendar events is partially implemented (1 WU remaining in backlog)

• Dark mode: Minor contrast issues with some card borders in certain browsers

Section 4 — Developer Documentation

Full developer documentation is in the repository README.md:

https://github.com/BarrettRyan-student/TestWebsite/blob/main/README.md

The README includes:

• Development tools: IDE (VS Code/Cursor), framework (Next.js 15), language (TypeScript), database (Supabase), testing (Playwright), deployment (Vercel), AI assistance (Cursor AI, Claude Code)

• Setup instructions: Clone, install, environment variables, database migrations, start dev server

• Workflow: Branching strategy (main + feature branches), PR process, issue tracking with GitHub Issues, commit conventions

• How to run: npm scripts (dev, build, start, lint, test), Playwright E2E test commands

• How to extend: Project structure overview, step-by-step guide for adding new features (database → API route → components → page → tests), API route template with auth pattern

• Deployment: Vercel auto-deploy from main, environment variable setup, Supabase auth configuration

Section 5 — User Documentation

User documentation is available in two locations:

1. In-App Help Page

Accessible at /help from any page in the app. Covers:

• What Anchor is and who it's for (Athlete vs Psychologist roles)

• Dashboard overview: calendar navigation, filter chips, Latest Check-In card

• Feature guide: Psychological State check-ins, Physical State logging, Sessions, Assessments

• Privacy & Safety statement

2. Landing Page

The public landing page at /landing provides a feature overview and role-based registration flow for new users, requiring no prior knowledge.

3. Known Limitations (disclosed to users)

• Anchor is not a medical or diagnostic tool — all data is self-reported

• Wearable sync is limited to OAuth connection; full data import is not yet available

• AI Coach responses depend on image quality; high-resolution screenshots produce better results

The documentation was updated after user testing to address questions about the emoji scale direction and the difference between check-ins and assessments.

Section 6 — Version Control (GitHub)

Repository: https://github.com/BarrettRyan-student/TestWebsite

Contribution Summary:

• Both team members (Ryan Barrett and Anthony Nania) contribute regularly with meaningful commit messages

• 40+ commits across the project lifetime

• Feature branches used for larger features (messages, wearables, goals) starting in Sprint 3

• Pull requests opened before merging significant features

• GitHub Issues used for bug tracking and feature requests

• Commit history reflects real development activity — feature implementation, bug fixes, dependency resolution, deployment management

Branch Hygiene:

• main branch is the production branch, deployed to Vercel on every push

• Experimental and completed feature branches have been cleaned up

• Pre-push checklist: run npm run build locally before pushing to catch type errors and hydration issues

Section 7 — Testing

Automated Testing Framework: Playwright (E2E browser testing)

Test Location: e2e/ directory in the repository

Test Coverage:

1. Authentication Tests (e2e/auth.spec.ts):

• Login page renders correctly (email, password fields, sign-in button)

• Invalid credentials show appropriate error message

• Successful login redirects to dashboard

• Protected routes redirect unauthenticated users to login

2. Navigation Tests (e2e/navigation.spec.ts):

• Dashboard loads with calendar and sidebar

• Navigate to Physical State page (verify Recovery, Training, Fueling tabs)

• Navigate to Psychological State page

• Navigate to Assessments page

• Navigate to Sessions page

• Physical State tab switching works (Recovery → Training → Fueling → Recovery)

3. Landing Page Tests (e2e/landing.spec.ts):

• Hero section and feature cards render

• Sign-in link navigates to login page

• Unauthenticated root (/) redirects to /landing

Why these tests matter:

These tests cover the core user workflow: an unauthenticated user lands on the site, gets redirected to the landing page, navigates to login, authenticates, and then accesses all major features. This is the primary path every user follows, and regressions here would block all functionality. The authentication tests were specifically informed by a hydration bug discovered in Sprint 3 that caused the login page to fail silently.

Running tests:

```
npx playwright install    # first time only — installs browser engines
```

npx playwright test # run all E2E tests

npx playwright test --ui # interactive UI mode

Portfolio Draft

Section 1: Program

Anchor is a mental and physical wellbeing tracking platform for sports psychologists and the athletes they serve. It consolidates psychological check-ins, physical recovery logs, training data, meal tracking, practitioner sessions, and trend charts into a single calendar-driven dashboard accessible from any browser.

Problem:

Sports psychologists have no purpose-built, lightweight tool for tracking athlete wellbeing between sessions. Practitioners rely on verbal reports, paper forms, and disconnected apps that produce no persistent, shared data.

Solution:

A role-aware web application where athletes self-report daily state in under 60 seconds, and psychologists can review a complete wellbeing timeline for any athlete in their caseload before a session.

Live Beta: https://anchor-testwebsite.vercel.app/

Target Users:

- Sports psychologists — review athlete data, manage caseloads, receive session context
- Athletes — log daily mood, physical state, meals, and medications; view personal trends

Core Features (Beta):

- Unified calendar dashboard with filter chips (Mood, Training, Recovery, Fueling, Assessments)
- Daily psychological check-in (Mood / Stress / Motivation 1–10 emoji scale + notes)
- Physical state logging: recovery, training load, fatigue, soreness, sleep quality
- Fueling log with Edamam food search API integration
- Medication tracking
- In-app threaded messaging with AI assistant
- Landing page with role-based registration flow
- Wearable integrations: Fitbit and Oura (authorize, sync)
- Role-based access: Psychologist athlete roster, drill-down, and attention scoring
- Trend charts: Mood, Recovery, Fueling consistency over time
- Settings: profile, height/weight, goals, notifications

Section 2: Teamwork

Member	Role	Primary Responsibilities

Ryan Barrett	Lead Developer	Architecture, full-stack feature development, deployment, debugging
Anthony Nania	Developer	Psychologist feature development, Supabase schema, QA/testing

Collaboration Practices:

- Weekly standups to report progress, blockers, and next commitments
- GitHub for version control with both members committing
- Feature branches introduced in Sprint 3 to reduce main-branch instability
- Pull requests opened before merging significant features (messages, wearables)
- Pair programming used when a team member encountered a blocker

Sprint 3 Retrospective Summary:

- Task ownership asymmetry (Ryan holding most implementation) creates a knowledge concentration risk. Sprint 4 will address this by assigning Anthony a primary feature from day one.
- Running npm run build locally before pushing would have caught the hydration bug earlier.
- Pair programming sessions were highly effective and will be used more proactively.

Section 3: Design Patterns

Repository / Service Layer Pattern

API routes in app/api/ act as a thin service layer between client components and Supabase. Components never call Supabase directly from the browser; they call Next.js API routes, which use the service-role key server-side. This centralizes data-access logic and keeps credentials off the client.

Role-Based Access Control (RBAC)

Middleware (middleware.ts) inspects the session and user role on every protected route before the page renders. Two roles — athlete and psychologist — receive different UI and different data access. Supabase Row-Level Security policies enforce the same boundaries at the database level, providing defense in depth.

Observer / Filter Pattern (Calendar)

The unified calendar uses a filter chip state (Training / Recovery / Mood / Fueling / Assessments). All components subscribe to the active filter set via React context; turning a filter off cascades through the calendar event list, Today panel, and sidebar card simultaneously without any component directly calling another.

Adapter Pattern (Wearable Integrations)

Fitbit and Oura have different OAuth flows and data schemas. Each integration has its own authorize/callback/sync route set. A common internal data shape (WorkoutLog, RecoveryLog) is produced by each adapter so the rest of the app remains agnostic to which wearable provided the data.

Strategy Pattern (AI Messaging)

The AI assistant is accessed through a route (/api/ai/) that can be swapped to a different model or provider without touching the messages UI. The response format is standardized so the front end only knows about message threads, not the underlying AI provider.

Section 4: Technical Writing

Documentation lives in two places:

In-app help:

/help page provides user-facing guidance on logging check-ins, reading the dashboard, and understanding the emoji scale.

Developer documentation:

Document	Purpose
README.md	App overview, roles, tech stack, local setup, deployment guide
docs/sprint-1.md	Sprint 1 goal, scope, acceptance criteria, file structure
docs/sprint-2.md	Sprint 2 goal, scope, acceptance criteria, technical notes
docs/roadmap.md	Full 6-sprint development plan with deliverables
docs/usability-test-playbook.md	Observation guide for usability testing
docs/portfolio-draft.md	Full project portfolio (this document)

Writing standards applied:

- Each sprint document leads with a SMART goal so any reader can verify completion
- Acceptance criteria are written as testable, user-visible outcomes (not internal implementation tasks)
- The README targets two audiences: a developer setting up locally, and a non-technical stakeholder who only needs the live URL and overview

Section 5: UML

Use Case Summary (by role):

Actor	Use Case
Athlete	Log daily check-in (Mood / Stress / Motivation)
Athlete	Log physical state (recovery, training load, meals, medication)
Athlete	Send message / chat with AI assistant
Athlete	View personal dashboard and trend charts
Athlete	Connect wearable (Fitbit / Oura)
Psychologist	View and manage athlete roster
Psychologist	Switch to athlete view and review their dashboard
Psychologist	Send messages to athlete
Psychologist	Assign athletes by email

System	Authenticate users via Supabase Auth (PKCE)
System	Enforce Row-Level Security on all database tables
System	Sync wearable data via OAuth callback

Component Overview:

Layer	Components
Pages (app/)	/landing, /login, /register, / (Dashboard), /psychological-state, /physical-state, /messages, /sessions, /assessments, /settings, /psychologist/*
API Routes (app/api/)	auth/*, checkins, recovery, workouts, meals, messages/*, ai, edamam/search, integrations/fitbit/*, integrations/oura/*
Middleware	Route protection + role enforcement (middleware.ts)
Database (Supabase)	profiles, check_ins, recovery_logs, workout_logs, meal_logs, medications, messages, psychologist_athletes, sessions
Auth	Supabase Auth — email + password, PKCE flow, session cookies

Section 6: Design Trade-offs

Supabase vs. Custom Backend

Chose Supabase. Less control over query optimization, but built-in auth, RLS, and instant Postgres with zero infrastructure management — appropriate for a two-person semester project.

Next.js App Router vs. Pages Router

Chose App Router. Server components reduce bundle size and are the modern Next.js paradigm, but introduced complexity (e.g., the hydration mismatch bug on the messages page).

localStorage vs. Supabase for Check-Ins

Started with localStorage in Sprint 2 for rapid prototyping; migrated to Supabase in Sprint 3 once auth was established so psychologists could view athlete data. Migration was smooth because check-in logic was isolated in lib/psych-checkins.ts.

Edamam API for Food Search vs. Manual Entry

Chose Edamam despite API dependency and rate limits. Search-and-select food logging is dramatically faster than manual entry, which is critical for daily compliance.

AI Assistant Embedded in Messages vs. Separate Tab

Embedded in messages. Less discoverable as a standalone feature, but athletes can reference recent logs in the same conversational space — more natural than switching between tabs.

Wearable Integration Depth

Built OAuth flow and sync scaffold, not full data processing. The auth handshake and data pipeline are established; processing wearable data into the app's schema is Sprint 4 work.

Section 7: Software Development Life Cycle Model

Model Used: Agile Scrum

Anchor is built using Scrum with 2-week sprints, a prioritized product backlog, weekly standups, and sprint retrospectives. This choice was driven by course requirements and by the nature of the problem — a client-facing wellbeing app where feature priorities can shift based on feedback.

Why Scrum fits this project:

- Product requirements were not fully defined at the start; Scrum allowed the backlog to evolve (e.g., Messages and AI were not in the original Sprint 3 plan but were added as high-value deliverables)
- Short sprints forced demo-ready code every two weeks, keeping the team accountable and giving the client a working product to evaluate at each checkpoint
- Retrospectives identified process issues (task imbalance, hydration bugs, pair-programming gaps) and fed improvements directly into the next sprint's working agreements

Scrum Artifacts in Use:

Artifact	Location
Product Backlog	This document (Part 1) + docs/roadmap.md
Sprint Backlog	This document (Part 1, Sprint Backlog table)
Burndown Chart	This document (Part 1, Burndown Chart table)
Sprint Retrospective	docs/Sprint 3 retrospective.docx
Definition of Done	Feature persists to Supabase, renders on mobile and desktop, does not break existing functionality

Alternative Considered — Kanban:

A Kanban flow (no sprints, continuous delivery) was considered and rejected because the course requires sprint-based submissions and a burndown chart, and because the fixed-deadline nature of each Beta/Alpha submission maps better to sprint timebox planning.

Note on Development Style:

The team flagged in the Sprint 3 retrospective that some implementation followed an exploratory, iteration-heavy style ('vibe coding'), especially for the AI integration and wearable OAuth flows. This is appropriate during a prototype phase but will be balanced with more upfront design in the final sprints to ensure code quality and maintainability.
